

Roteiro de Apresentacao - Counting Sort

Comparacao entre Python e Rust

Slide 1 - Titulo

Texto no slide:

Analise Experimental do Counting Sort

Comparacao entre Python e Rust

Equipe 7

Fala:

O objetivo do projeto foi implementar e analisar o Counting Sort em duas linguagens, comparando os resultados praticos com a complexidade teorica.

Slide 2 - Objetivo

Texto no slide:

- Implementar o Counting Sort em Python e Rust
- Medir o tempo de execucao em diferentes cenarios
- Comparar os resultados com $\Theta(n + k)$
- Avaliar diferencas praticas entre as linguagens

Fala:

O trabalho busca entender se os resultados experimentais acompanham a complexidade teorica e como a escolha da linguagem afeta o tempo real de execucao.

Slide 3 - Funcionamento do Counting Sort

Texto no slide:

- Conta a frequencia de cada valor
- Usa um vetor auxiliar de contagem
- Reconstrui o vetor ordenado

Fala:

Diferente de algoritmos como Quick Sort ou Merge Sort, o Counting Sort nao compara elementos entre si. Ele usa o intervalo dos valores para contar ocorrencias e reconstruir a saida ordenada.

Slide 4 - Complexidade

Texto no slide:

Complexidade temporal: $\Theta(n + k)$

n = quantidade de elementos

k = intervalo de valores possiveis

Fala:

No Counting Sort, a ordem inicial do vetor influencia pouco o custo. O fator mais importante, alem da quantidade de elementos, e o tamanho do intervalo k .

Slide 5 - Metodologia Experimental

Texto no slide:

n fixo: 1.000.000 elementos

Cenários:

- $k = 100$
- $k = 1.000.000$
- $k = 100.000.000$

30 execuções por cenário

Fala:

Como o Counting Sort depende de $n + k$, os cenários foram definidos mantendo n fixo e variando k . Assim, avaliamos diretamente o impacto do intervalo de valores no desempenho.

Slide 6 - Validacao das Entradas

Grafico sugerido:

analise/graficos/00_verificacao_checksums.png

Texto no slide:

- Python e Rust usaram as mesmas entradas
- Entradas geradas por seed
- Checksums comparados
- 90/90 checksums iguais

Fala:

Para garantir uma comparacao justa, as duas linguagens reconstruiram os vetores a partir das mesmas sementes. Os checksums confirmaram que as entradas foram equivalentes em todas as execucoes.

Slide 7 - Resultados de Tempo

Grafico sugerido:

analise/graficos/04_cenarios_com_desvio.png

Texto no slide:

- O tempo aumenta conforme k cresce
- O pior cenario ocorre com $k = 100.000.000$
- Rust teve menor tempo em todos os cenarios

Fala:

Os resultados mostram que o aumento de k impacta diretamente o tempo de execucao. Isso acontece porque o algoritmo precisa criar e percorrer um vetor de contagem proporcional ao intervalo de valores.

Slide 8 - Teoria vs Pratica

Grafico sugerido:

analise/graficos/02_real_vs_teorico_por_k_log.png

ou

analise/graficos/03_real_vs_teorico_tres_cenarios.png

Texto no slide:

- Curva teorica: $c * (n + k)$
- Dados reais acompanham o crescimento esperado
- Escala log facilita a visualizacao

Fala:

A curva teorica ajustada acompanha a tendencia dos dados reais. Como n permanece constante, o crescimento observado esta associado principalmente ao aumento de k .

Slide 9 - Comparacao Python vs Rust

Grafico sugerido:

analise/graficos/05_speedup_rust_vs_python_cenarios.png

Texto no slide:

Speedup:

- $k = 100$: 71,1x
- $k = 1.000.000$: 16,5x
- $k = 100.000.000$: 34,0x

Fala:

O objetivo nao e dizer que a complexidade muda entre as linguagens. A complexidade teorica e a mesma, mas a implementacao e o modelo de execucao de cada linguagem afetam o tempo real.

Slide 10 - Velocidade de Processamento

Grafico sugerido:

analise/graficos/06_velocidade_linguagens_cenarios.png

Texto no slide:

- Mede elementos processados por segundo
- Rust manteve maior taxa de processamento
- A taxa cai quando k aumenta

Fala:

A taxa de processamento mostra a diferenca pratica entre as linguagens. Mesmo com a mesma complexidade, Rust processou mais elementos por segundo nos cenarios testados.

Slide 11 - Conclusao

Texto no slide:

- Counting Sort segue $\Theta(n + k)$
- O crescimento de k impacta tempo e memoria
- Rust foi mais rapido nos testes
- A escolha do algoritmo depende da relacao entre n e k

Fala:

O Counting Sort e eficiente quando k e pequeno em relacao a n . Quando o intervalo de valores cresce muito, o custo de memoria e tempo aumenta, reduzindo a vantagem pratica do algoritmo.

Versao reduzida, se precisar de menos slides:

1. Titulo
2. Algoritmo e complexidade
3. Metodologia
4. Validacao por checksums
5. Resultados de tempo
6. Teorico vs real
7. Python vs Rust

8. Conclusao